

Moed B 2019 – Solution

September 3, 2019

Question 1

There are many solutions for this question. We present the ones that most of the accepted solutions are based on.

- (a) 1. A Regex over $\Sigma = \{a, b\}$:
- First Regex: $(b \cup (ab^*a))^*$
 - Second Regex: $(b(ab^*a)^*)^*$
2. A CFG with a single variable S and the following rules:
- First CFG: $S \rightarrow bS \mid aSaS \mid \epsilon$
 - Second CFG: $S \rightarrow bS \mid aSa \mid SS \mid \epsilon$
- (b) **The claim is correct.** We accepted solutions for this question. We present here two of them. The first solution is based on building a DFA recognizing $\text{longest}(L)$, and the second follows by bounding the number of the Myhill-Nerode equivalence classes for $\text{longest}(L)$.

Solution 1 Assume that $L \in \text{REG}$, and let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ be a DFA for L . To build an automaton for $\text{longest}(L)$, we note that a word w is in $\text{longest}(L)$ if the run of $\mathcal{A}$ on w is accepting, and it cannot be extended to a longer accepting run. So a DFA for $\text{longest}(L)$ is defined on top of $\mathcal{A}$'s structure, and differs only in the set of accepting states. Formally, we define $\mathcal{B} = \langle Q, \Sigma, \delta, q_0, F' \rangle$, where $F' = \{q \in F \mid \forall z \in \Sigma^+, \delta^*(q, z) \notin F\}$. That is, a state q of $\mathcal{B}$ is accepting if it is an accepting state of $\mathcal{A}$, and no accepting state of $\mathcal{A}$ is reachable from it. We show that $L(\mathcal{B}) = \text{Longest}(L(\mathcal{A}))$. Consider a word $w \in \Sigma^*$. Then,

$$\begin{aligned}
 w &\in L(\mathcal{B}) \\
 &\text{iff } \delta^*(q_0, w) \in F' \\
 &\text{iff } \delta^*(q_0, w) \in F, \text{ and } \forall z \in \Sigma^+, \text{ it holds that } \delta^*(\delta^*(q_0, w), z) \notin F \\
 &\text{iff } w \in L(\mathcal{A}), \text{ and } \forall z \in \Sigma^+, \text{ it holds that } \delta^*(q_0, w \cdot z) \notin F \\
 &\text{iff } w \in L(\mathcal{A}), \text{ and } \forall z \in \Sigma^+, \text{ it holds that } w \cdot z \notin L(\mathcal{A}) \\
 &\text{iff } w \in \text{longest}(L(\mathcal{A})).
 \end{aligned}$$

Note that we have relied on the fact¹ that $\delta^*(q, w \cdot z) = \delta^*(\delta^*(q, w), z)$ for every state q , and words w and z . Also, note that F' is well defined, and thus, so is $\mathcal{B}$. Hence, $\text{longest}(L)$ is regular as there exists a DFA for it.

Solution 2 As we know, the Myhill-Nerode equivalence classes of a language L partition the set Σ^* . Thus, to show that $\text{longest}(L)$ is regular, it is sufficient to show that the partition induced by $\text{longest}(L)$ is coarser than the partition induced by L . Namely, we show that the following holds. For every two words $x, y \in \Sigma^*$, if $x \sim_L y$, then $x \sim_{\text{longest}(L)} y$. Let $z \in \Sigma^*$ be such that $xz \in \text{longest}(L)$ and $yz \notin \text{longest}(L)$, that is, z is a separating suffix between x and y w.r.t $\text{longest}(L)$. We show that $x \not\sim_L y$. Note that as $xz \in \text{longest}(L)$, we have that $xz \in L$ and for all

¹You have seen this in class by a simple induction

$z' \in \Sigma^+$, $xzz' \notin L$. As $yz \notin \text{longest}(L)$, one of the following holds. Either $yz \notin L$ in which case z also separates between x and y w.r.t L , or the case is that $yz \in L$ and there is some z' such that $yz' \in L$ in which case zz' separates between x and y w.r.t L .

(c) **The claim is incorrect.**

Consider the language $L = \{1^p : p \text{ is a prime number}\}$ over $\Sigma = \{1\}$. As seen in class, L is not regular. Therefore, it is left to show that $\text{longest}(L)$ is regular. We show that $\text{longest}(L) = \emptyset$. Assume by the way of contradiction that $\text{longest}(L) \neq \emptyset$, and let $w \in \text{longest}(L)$. Then, $w \in L$, and for every word $z \in \Sigma^+$, it holds that $w \cdot z \notin L$. Implying that $w = 1^p$ for a prime number p , and for every $k \geq 1$, it holds that $p + k$ is not a prime number. Hence, there is a finite number of prime numbers, and so we have reached a contradiction.

Question 2

(a) **The claim is correct.**

We saw in class that the existence of an enumerator for a language L and the fact that $L \in RE$ are equivalent. Now, from the existence of the enumerator E_1 , we get that $L \in RE$. From the existence of the enumerator E_2 , we get that $\bar{L} \in RE$. From the definition of $coRE$, we get that $L \in co-RE$. Overall, we get that $L \in RE$ and $L \in coRE$, so $L \in RE \cap coRE$. We saw in class that $R = RE \cap coRE$, so we conclude that $L \in R$.

(b) **The claim is correct.**

Assume by the way of contradiction that there is a TM M that decides B . To get a contradiction, we show that the existence of M implies the existence of a TM M' that decides A . The machine M' is defined as follows. Given an input $w \in \Sigma^*$:

- (a) If $|w|$ is odd, simulate M on w and answer the same.
- (b) If $|w|$ is even, simulate M on w and answer the opposite (that is, if M accepts, then reject, and if M rejects, then accept).

Since M is a decider, M' is also a decider. This is because all it does is simulating M after checking the parity of the length, which is a simple procedure that always halts.

We show that that M recognizes A . For the first direction, consider a word $w \in A$. If the length of w is odd, then $w \in B$, and therefore M accepts it. Since M' answers as M on inputs of odd lengths, M' accepts too, and we are done. If the length of w is even, then $w \notin B$, and therefore M rejects it. Since M' answers the opposite of M on inputs of even lengths, M' will accept. Conversely, if $w \in L(M)$, then either the length of w is odd and M accepts w , or the length of w is even and M rejects w . In both cases, as M decides B , we have that $w \in L$, so we are done.

(c) **The claim is correct.**

We show that $ALL_{TM} \leq_m L$. Since $ALL_{TM} \in \overline{RE \cup coRE}$, we conclude using the reduction theorem that $L \in \overline{RE \cup coRE}$.

Construction: Given the input $\langle M \rangle$, which encodes a TM, the reduction outputs $\langle M' \rangle$, where M' is a TM that works as follows: Given an input word x , it simulates the run of M on x . If M accepts, then so does M' . If M rejects, then M' doesn't halt by repeatedly writing some letter on the tape and moving to the right.

Correctness: If $\langle M \rangle \in ALL_{TM}$, then M accepts every word and so does M' , and in particular it halts on every word. Thus, $\langle M' \rangle \in L$. Conversely, If $\langle M \rangle \notin ALL_{TM}$, then there exists a word x that is not accepted by M . So, M either rejects x or does not halt on it. In both cases, M' does not halt on x . Thus, $\langle M' \rangle \notin L$.

Computability: Similarly to reductions we've seen in class, M' can be a universal TM with the additional mentioned subprocedure of writing letters, so it is computable.

Question 3

(a) **The claim is correct.**

Let $L \in NP$, and let K be a non-trivial language. Assuming $P = NP$, it holds that $L \in P$. Thus, there exists a TM M that decides L in polynomial time. Since K is not trivial, there are $w \in K$, and $w' \in \Sigma^* \setminus K$. Consider the following reduction from L to K . On input x , the reduction simulates the run of M on x . If M accepts, the reduction returns w . Otherwise, the reduction returns w' . Clearly, this reduction is correct. Since M runs in polynomial time, the reduction also runs in polynomial time. Thus, for every $L \in NP$, we have that $L \leq_p K$. Hence K is NP-Hard. Since, in addition, $K \in NP$, we have that K is NP-complete.

(b) **The language is in P.**

A TM M that decides A in polynomial time proceeds as follows. On input w :

- (a) Check whether w is a Boolean formula in CNF. If not, reject. Otherwise, let φ denote this formula.
- (b) Let $X = \{x_1, \dots, x_n\}$ be the set of variables in φ . Let $S_1, S_2, \dots, S_k$ be a sequence of all subsets of size 10 of X , ordered by some order that M can generate in polynomial time, for example a lexicographic order on $\{1, \dots, n\}$. Each set $S \subseteq X$ induces an assignment f to X in which $f(x_j) = T$ iff $x_j \in S$. Go over all the sets in the sequence. If φ is satisfied under one of the induced assignments, accept. Otherwise, reject.

Correctness: We check all relevant assignments, and we accept iff one of them satisfies φ .

Complexity: The number of sets in the sequence is $\binom{n}{10} = \frac{n!}{10!(n-10)!} \leq n^{10}$, and hence is polynomial in n and also in w . In addition, checking each assignment is linear in φ , so the total run time is polynomial.

(c) **The language is NP-complete.**

$A \in NP$:

A verifier V receives a formula φ and an assignment S to its variable. The verifier V tests whether φ has a CNF-form, whether the number of variables assigned *TRUE* is between $\frac{n}{3}$ and $\frac{2n}{3}$, and whether S satisfies φ . If all three tests pass, then V accepts. Otherwise, it reject. Since all tests can be performed in polynomial time, the verifier V runs in polynomial time.

$A \in NP\text{-HARD}$:

We show that $SAT \leq_p A$. As $SAT \in NP\text{-HARD}$, we conclude that $A \in NP\text{-HARD}$.

Construction: On input φ with n variables, the reduction generates φ' , which is $\varphi \wedge \varphi_T \wedge \varphi_F$. The formula φ_T is a formula in CNF with n clauses of the form $(x_i \vee x_i \vee x_i)$, for $i \in [n]$, where x_i are new variables. Similarly, φ_F is a formula in CNF with n clauses of the form $(\overline{y_i} \vee \overline{y_i} \vee \overline{y_i})$, for $i \in [n]$, where y_i are new variables.

Correctness: If φ has a satisfying assignment to the n original variables, then we can extend it to a new assignment that satisfies φ' by setting all the x_i variables to *TRUE* and all the y_i to *FALSE*. The new assignment has at least a third of the variables assigned *TRUE* (the new x_i 's) and at least a third of the variables assigned *FALSE* (the new y_i 's). The new assignment satisfies φ' as the original assignment satisfies φ , and it easy to verify that the assignment to the x_i and y_i satisfies φ_T and φ_F . Therefore, $\varphi' \in A$. Conversely, if $\varphi' \in A$, then it has a satisfying assignment. That assignment restricted to the original variables satisfies φ . Hence $\varphi \in SAT$.

Complexity: The construction clearly takes polynomial time.